

Законы развития технических систем - объективные функции развития, выявленные на основе анализа патентов и истории техники, которые описывают как эволюционируют технические системы независимо от воли отдельных изобретателей

Основные группы

1. Статика (условия формирования и развития тех. систем)
2. Кинематика (как будет развиваться на следующем этапе)
3. Динамика (конкретные пути развития)
4. Применение (как правильно применять эти законы)

Чем полезно для программистов?

- Прогнозирование развития технологий
- Выбор архитектурных решений
- Системное мышление
- Оптимизация существующих систем

Закон полноты составных частей системы

Для функционирования технической системы обязательно наличие следующих составных частей и их минимальная работоспособность

Составные части технической системы

1. Двигатель
2. Трансмиссия
3. Рабочий орган
4. Орган управления

Значение

1. Отсутствие/неработоспособность делают систему нежизнеспособной
2. Указывает слабые места системы
3. Эволюция системы начинается с формирования всех 4-х компонентов
4. ...

Закон полноты в программной инженерии:

- БД - **двигатель**
- Бизнес логика - **трансмиссия**
- Интерфейс - **рабочий орган**
- Система управления - **орган управления**

Закон энергетической проводимости

Для работы технической системы необходим сквозной проход энергии через все части системы. Эволюция технических систем направлена на улучшение энергетической проводимости - устранение потерь, повышение КПД

Закон энергетической проводимости в программной инженерии

В программных системах энергией является **информация и данные**

- Источники данных (БД, API, файловые системы)
- Обработка данных (бизнес-логика, алгоритмы, преобразование форматов)
- Представление данных (UI, API-ответы, отчеты)

Закон согласования ритмики частей системы

Необходимым условие жизнеспособности технической системы является согласование ритмики

Согласно закону

1. Все части системы должны работать в едином ритме
2. Без согласования ритмики происходит снижение эффективности
3. Согласование может достигаться как синхронизацией, так и целенаправленной асинхронностью (буферизацией)

Значение

- выявляет проблемы согласования частей системы
- прогнозирование направления развития системы
- указывает на необходимость согласования частот

Закон согласования ритмики в программной инженерии

1. **Проблемы рассогласования ПО** Гонка процессов, взаимные блокировки
2. **Методы согласования ритмики в ПО** Синхронизация, мьютексы, буферизация, очереди, кэширование, транзакции, асинхронность
3. **Эволюция согласования** От блокирующих операций в неблокирующим, от монолитной обработки в параллельной, от жесткой синхронизации к гибким механизмам

Закон повышения идеальности

Все технические системы в ходе эволюции стремятся в идеальности

Формула

$$\text{Идеальность} = \frac{\sum \text{Полезные функции}}{\sum \text{Факторы расплаты}}$$

Пути повышения идеальности

1. Увеличение полезных функций (повышение качества функций, создание новых функций, расширение области применения)
2. Уменьшение факторов расплаты (снижение энергопотребления, удешевление производства, упрощение конструкции)

Идеальная система - та, которой нет, но ее функция выполняется

$$\text{Идеальность ПО} = \frac{\text{Функциональность} + \text{Удобства} + \text{Надежность}}{\text{Сложность} + \text{Ресурсоемкость} + \text{Стоимость разработк}}$$

Закон перехода в надсистему

Исчерпав возможности развития система включается в надсистему в качестве одной из частей, при этом дальнейшее развитие идет на уровне надсистемы

Согласно закону

1. Система становится частью надсистемы, приобретая новые функции и возможности
2. Когда система достигает предела развития, дальнейшая эволюция происходит через объединение с другими системами

3. Переход в надсистему может происходить через объединение однородных или разнородных систем

Переход в надсистему - качественный скачок

Закон перехода в надсистему в программной инженерии

- Отдельные программы -> Интегрированные пакеты -> Экосистемы
- Локальные файлы -> БД -> Data Lakes
- Файловый обмен -> API
- Серверы - Виртуализация - Облака - Мультиоблака

Современные тенденции перехода в надсистему в IT

1. Технологические (IoT, цифровые экосистемы)
2. Организационные (DevOps, Agile-трансформация)

Что дает?

- Расширение функциональности
- Отказоустойчивость
- Вынесение ненужных функций
- ...

Примеры применения ТРИЗ в программной инженерии

1. **Оптимизация ПО** Проблема: высокая сложность
2. **Структурное программирование** Проблема: спагетти-код
3. **Оптимизация интерфейса** Проблема: количество информации и ее доступность

4. **Решение проблем утечек памяти** *Проблема:* утечки памяти в долго работающих приложениях
5. **Оптимизация загрузки веб-приложения** *Проблема:* противоречие между функциональностью веб-приложения и скоростью загрузки
6. **Самооптимизирующийся код** *Проблема:* необходимость постоянной оптимизации кода для различных условий
7. **Эволюция облачных вычислений** *Проблема:* необходимость эффективного использования вычислительных ресурсов

Применение ТРИЗ для оптимизации архитектуры микросервисов

| Табличка [40principles.pdf](#) (первые 4)